



Vectra XDR: Google SecOps SIEM Integration - User Guide

V 1.0.0

Contents

Contents	2
Overview	4
Vectra Platform	4
Google SecOps	4
Looker Dashboards for Vectra RUX	4
Release Notes	5
App Installation & Configuration	5
Pre-Requisites	5
Creating zip of the cloud function	6
Using Secrets	7
Add the Vectra API Secret in Secret Manager	7
Create a GCP Bucket	8
Service Account of Google SecOps Account	9
Cloud Function Deployment	10
Configure Scheduler	13
Parser Deployment	14
View Events in Google SecOps	15
Environment Variables to be configured in Cloud Function	16
Looker Installation & Configuration	18
Pre-Requisites	18
User Permissions	18
Create a connection to Google SecOps in Looker	18
Get the Block from GitHub Repository	20
Dashboards	27
Entities Dashboard	27
1. Filters description as per label	27
2. Panel description as per label	27
Detection Dashboard	29
1. Filters description as per label	29
2. Panel description as per label	29
Audit Dashboard	30
1. Filters description as per label	30
2. Panel description as per label	30
Lockdown Dashboard	31
1. Filters description as per label	31
2. Panel description as per label	31
Health Dashboard	32
1. Panel description as per label	32
Notes	33

Limitations.....	33
Troubleshooting.....	35
GCP Resources/Services Approximate Cost Details.....	37
References.....	38

Overview

Vectra Platform

Vectra AI delivers an AI-driven hybrid attack detection, investigation and response platform. The Vectra AI Platform is the integrated signal powering XDR providing hybrid attack surface coverage across identity, public cloud, SaaS, and data center networks; AI-driven Attack Signal Intelligence that prioritizes real attacks in real-time and integrated, automated, and managed response services.

Google SecOps

Google SecOps is a cybersecurity telemetry platform for threat hunting, and threat intelligence and is part of the Google Cloud Platform. Google SecOps stores log events it receives in two formats: either as the original raw log or structured Unified Data Model (UDM) log. There are two critical elements to consider for parsing, Unified Data Model (UDM) which defines the schema for parsing, and Configuration Based Normalizers (CBN) which describes how log data is transformed to the UDM schema.

Looker Dashboards for Vectra RUX

This integration aims to enable seamless ingestion, parsing, and visualization of Vectra network intelligence data within Google SecOps SIEM. This integration will allow Google SecOps SIEM to receive real-time detections, hosts, accounts, health, audit, and lockdown data from Vectra using Vectra API , enriching the SIEM's threat detection and response capabilities with comprehensive network data.

Release Notes

V1.0.0

- Provided the ingestion script which deployed as a GCP cloud function to collect data from the Vectra Platform and ingest in the Google SecOps.
- Provided the parser that processes data ingested from the Vectra platform and converts it into the Google SecOps UDM data model.
- Provided below dashboards for visualization
 - Entities
 - Detection
 - Audit
 - Health
 - Lockdown

App Installation & Configuration

Pre-Requisites

- Google SecOps console.
- Vectra credentials (API Client, API Secret, Vectra Portal URL)
- GCP Project with the below required permissions:
 - GCP user and project service account should have owner permissions.
- GCP Services
 - Cloud function (4-core CPU or higher is recommended for cloud function configuration)
 - GCS bucket
 - Secret Manager
 - Cloud Scheduler
- Looker Instance

Creating zip of the cloud function

- You can download the zip directly from the [Vectra GitHub Repo](#) from the vectra_rux_connector branch.
- OR
- Create a zip file with the contents of the following files:
 1. Download the common directory from Git [repository](#).
 2. Download the contents of the ingestion script. (Link will be added here once it will be submitted)
 3. Create a zip in the following structure.

❖ Example:

```
|— README.md
|— common
|   |— __init__.py
|   |— auth.py
|   |— auth_test.py
|   |— env_constants.py
|   |— ingest.py
|   |— ingest_test.py
|   |— status.py
|   |— utils.py
|   |— utils_test.py
|— .env.yml
|— constant.py
|— exception.py
|— main.py
|— main_test.py
|— requirements.txt
|— utils.py
|— utils_test.py
|— vectra_client.py
|— vectra_client_test.py
```

Using Secrets

- [Environment variables](#) marked as secret must be configured as secrets on Google Secret Manager. [\[REF\]](#)
- Once the secrets are created on Secret Manager, use the secret's resource ID as the value for environment variables.

For example:

CHRONICLE_SERVICE_ACCOUNT:

projects/{project_id}/secrets/{secret_id}/versions/{version_id}

Add the Vectra API Secret in Secret Manager

1. Log in to the "<https://console.cloud.google.com/>" using valid credentials.
2. Navigate to 'Secret Manager'.
3. Click on 'Create Secret'.
4. Provide the name for the secret in the 'Name' field.
5. Add your Vectra API Secret value to the Secret Value.
6. Keep the other configurations as default, Click on the 'Create Secret' button.

Similarly, Create a secret for Vectra **Client ID** and **Chronicle Service Account**.

[←](#) Create secret

Secret details

This will create a secret with the secret value in the first version. [Learn more](#)

Name *
vectra_client_secret

The name should be identifiable and unique within this project.

Secret value
Input your secret value or import it directly from a file.

Upload file [BROWSE](#)

Maximum size: 64 KiB

Secret value
secret_value

CRC-32C checksum: 0x6BBC2F7C

[CREATE SECRET](#) [CANCEL](#)

Refer to [this](#) page for more information about how to create secrets.

Once the secrets are created on Secret Manager, use the secret's resource id as the value for environment variables. For example

SECRET_KEY: projects/{project_id}/secrets/{secret_id}/versions/{version_id}

Note : Ensure that the resource ID format matches the following structure: projects/{project_id}/secrets/{secret_id}/versions/{version_id}. You can copy the resource name from the secret version details to obtain this value.

Create a GCP Bucket

1. Log in to the "<https://console.cloud.google.com/>" using valid credentials.
2. Navigate to Buckets in GCP.
3. Click on the Create button.
4. Enter the name of the bucket.
5. Users can select the region and modify the optional parameters if required and then click on the Create button.

Copy the bucket name and provide it in the GCP_BUCKET_NAME environment variable.

← Create a bucket

- **Get Started**

Pick a **globally unique, permanent name**. [Naming guidelines](#)

Tip: Don't include any sensitive information

Optimize storage for data-intensive workloads



Labels (optional)



CONTINUE

- **Choose where to store your data**

Location: us (multiple regions in United States)

Location type: Multi-region

Service Account of Google SecOps Account

1. Log in to the "<https://console.cloud.google.com/>" using valid credentials.
2. From the project selector menu, choose the GCP project where your Google SecOps is running.
3. Navigate to Service Accounts.
4. Click on Create Service Account.
5. Provide a name for the service account (e.g., chronicle_test).
6. Click Create and Continue.
7. In the Grant this service account access to the project section, assign the role **Owner**.
8. Click Done.
9. Generate and Download the Service Account Key
 - a. After successfully creating the service account, open the service account details.
 - b. Go to the Keys tab and click Add Key.
 - c. Select Create New Key and choose the key type as JSON.
 - d. The service account key will be downloaded to your system. This will be used in the subsequent steps.

The image shows two screenshots from the Google Cloud Console. The top screenshot is the 'Create service account' wizard, which is divided into two columns. The left column contains steps 1 and 2, and the right column contains steps 3 and 4. Step 1, 'Service account details', includes fields for 'Service account name' (chronicle_test), 'Service account ID' (chronicle-test), 'Email address', and 'Service account description'. Step 2, 'Grant this service account access to project (optional)', shows the 'Role' set to 'Owner' and an 'IAM condition' option. The bottom screenshot shows the 'Keys' tab of a service account named 'p4.iam.gserviceaccount.com'. It displays a warning about security risks and a table with one key entry that is 'Active'. A modal dialog is open over the 'ADD KEY' button, showing options for 'Key type' (JSON or P12) and a 'CREATE' button.

← Create service account

1 Service account details

Service account name
chronicle_test

Display name for this service account

Service account ID *
chronicle-test

Email address:

Service account description

Describe what this service account will do

CREATE AND CONTINUE

2 Grant this service account access to project (optional)

Grant this service account access to Google-SecOps-SIEM-Integration so that it has permission to complete specific actions on the resources in your project. [Learn more](#)

Role
Owner

IAM condition (optional) [+](#) ADD IAM CONDITION

Full access to most Google Cloud resources. See the list of included permissions.

+ ADD ANOTHER ROLE

CONTINUE

3 Grant users access to this service account (optional)

DONE CANCEL

4 Grant users access to this service account (optional)

DONE CANCEL

logging-quickstart@p4.iam.gserviceaccount.com

DETAILS PERMISSIONS KEYS METRICS LOGS

Keys

Service account keys could pose a security risk if compromised. [service accounts on Google Cloud](#)

Google automatically disables service account keys detected as unused.

Add a new key pair or upload a public key certificate from an existing key.

Block service account key creation using [organization policies](#).

[Learn more about setting organization policies for service accounts](#)

ADD KEY

Type Status Key

Active

p4.iam.gserviceaccount.com

Downloads a file that contains the private key. Store the file securely because this key can't be recovered if lost.

Key type

☒ JSON
Recommended

☐ P12
For backward compatibility with code using the P12 format

CANCEL CREATE

Cloud Function Deployment

1. Log in to the "<https://console.cloud.google.com/>" using valid credentials.
2. Navigate to the 'Cloud Functions'
3. Click on Create Function. Set the configuration section. Add environment, function name, region.
 - a. Select Cloud Run function in the Environment dropdown.
 - b. Enter the unique function name and select your region from the region dropdown
 - c. Keep Require Authentication as selected in the Trigger section.
4. Click on the dropdown for Runtime, build, connections, and security settings.
5. Select the below options in the RUNTIME.
 - a. Memory allocated - 8 GiB (Recommended)
 - b. CPU (preview) - 4 (Recommended)
 - c. Timeout - 3600 seconds
 - d. Keep autoscaling configurations as default.
 - e. Concurrency - 1
 - f. Service account - Select your current GCP project service account
 - g. Click on 'Add Variables' and add the [environment variables](#). Ensure you include the secret resource ID for any variable where 'Secret' is set to 'Yes'.
6. Click on next and Go to the Code upload section.
7. Select Python 3.12 in the Runtime dropdown.
8. Select ZIP Upload from the Source code.
9. Keep the entry point as **main**.
10. Select the created bucket in the Destination bucket dropdown.
11. Browse and select the downloaded application zip.
12. Click on the Deploy button.
13. After a few minutes, the cloud function will be deployed successfully.

[←](#) Create function

1 Configuration — 2 Code

Basics

Environment
Cloud Run function

Function name *
vectra_function

Region *
us-central1 (Iowa)

Trigger

Trigger type
HTTPS

URL
https://cloudfunctions.net/vectra_function

Authentication ?
☐ Allow unauthenticated invocations
Check this if you are creating a public API or website.
☒ Require authentication
Manage authorized users with Cloud IAM.

Runtime, build, connections and security settings

< RUNTIME BUILD CONNECTIONS SECURITY AND >

Memory allocated *
8 GiB

CPU *
4

Timeout *
3600 seconds

Concurrency
Maximum concurrent requests per instance
1

Autoscaling ?

Minimum number of instances
0

Maximum number of instances
100

Runtime service account ?

Service account
Default compute service account

By default Cloud Functions uses the automatically created Default Compute Engine Service Account. [Learn more about service accounts.](#)

Runtime environment variables ⓘ

Name 1 * VECTRA_PORTAL_URL	Value 1 https://1111.cc1.portal.vectra.ai
Name 2 * CLIENT_ID	Value 2 projects/11111/secrets/client_id_pi
Name 3 * SECRET_KEY	Value 3 projects/11111/secrets/client_secr
Name 4 * CHRONICLE_CUSTOMER_ID	Value 4 xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxx
Name 5 * GCP_PROJECT_NUMBER	Value 5 xxxxxxxxxxxxxx
Name 6 * CHRONICLE_REGION	Value 6 us
Name 7 * CHRONICLE_SERVICE_ACCOUNT	Value 7 projects/xx/secrets/chronicle_servi
Name 8 * GCP_BUCKET_NAME	Value 8 vectra_bucket_name
Name 9 * HISTORICAL	Value 9 false

+ ADD VARIABLE

NEXT CANCEL

Configure Scheduler

1. Log in to the "<https://console.cloud.google.com/>" using valid credentials.
2. Navigate to 'Cloud scheduler'.
3. Click on 'Create a Job'.
4. Enter a unique name for the scheduler and select your region in Region.
5. Enter the unix-cron format for the Frequency, when the scheduler job should invoke. Sample value(*/20 * * * *). It is recommended to set the frequency to a minimum of 5 minutes and not less, as there are five types of events being collected and ingested.
6. Select the timezone and click on 'Continue'.
7. Select the target type as "HTTP".
8. In the GCP Cloud interface, navigate to created cloud function details and under trigger sections and copy the trigger URL of the cloud function that the user creates a scheduler and paste it into the URL.
9. Keep POST as it is in the HTTP method.
10. Select "Add OIDC Token" in Auth Header.
11. In the service account field, select your current GCP project service account.
12. Enter **30m** in the Attempt deadline config.
13. Click on the 'Create' button.

Cloud function will be executed as per the frequency provided in the Cloud Scheduler.

Cloud Scheduler / Jobs / Edit job: Vectra_Schedule

Vectra_Schedule

• Define the schedule

Region

us-central1

Description

Vectra Schedule

Frequency *

*/05 * * * *

Schedules are specified using unix-cron format. E.g. every minute: '* * * * *', every 3 hours: '0 */3 * * *', every Monday at 9:00: '0 9 * * 1'. [Learn more](#)

• Minute:

◦ Every 5 minutes

Timezone *

India Standard Time (IST)

Jobs in set in timezones affected by Daylight Saving Time can run outside of cadence during DST change. Using a UTC timezone can avoid the problem. [Learn more](#)

CONTINUE

• Configure the execution

• Configure optional settings

• Configure the execution

Target type *

HTTP

URL *

https:// cloudfunctions.net/vectra_client

HTTP method

POST

HTTP headers

Some headers are set to default values or removed by Cloud Scheduler. [Learn more](#)

+ ADD A HEADER

Body

Auth header

Add OIDC token

Service account *

Default compute service account

This service account must have permission to invoke the target. For example, the Cloud Functions Invoker role is required to schedule a Cloud Function. [Learn more](#)

Audience

https:// cloudfunctions.net/vectra_client

Audience limits recipients for the OIDC token. Typically, the job's target URL (without any url parameters). If not specified, by default, Cloud Scheduler will use the whole URL, including request parameters, as the Audience.

CONTINUE

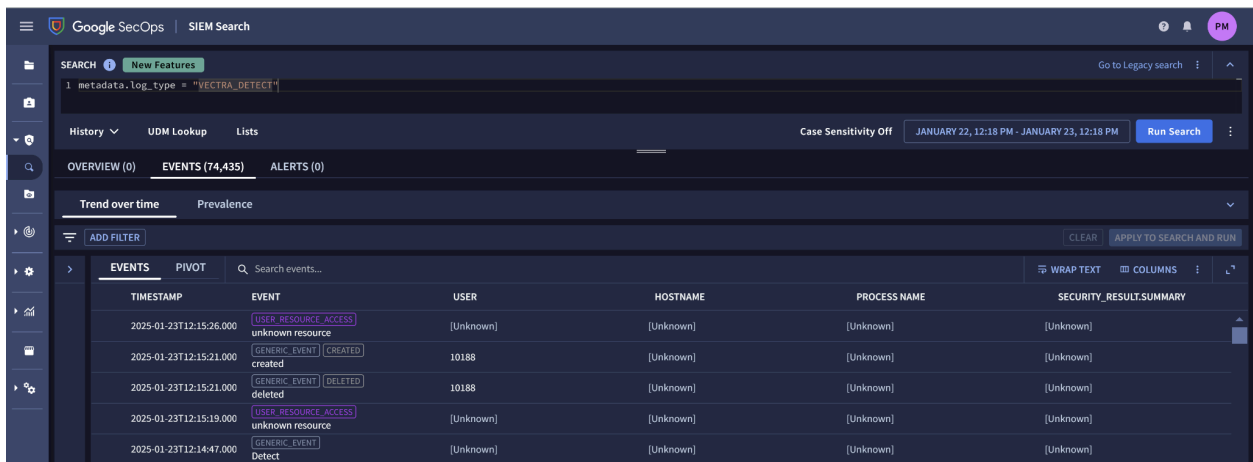
Parser Deployment

1. In the navigation bar, select **Setting > SIEM Settings > Parsers**.
2. Search the **Parsers** table for the log type you want to extend.
3. Click **Create Parser**.
4. Select **Vectra Detect** log source from the **Log Source** list.
5. Select **Start with Raw Logs** Only to create a new parser.
6. Copy the parser content from github repo and paste it in the code section.
7. Click **Validate** to validate the custom parser, the validation process may take a few minutes.
8. Click **Submit**.
9. After successful validation, click on **Next**
The parser is picked for normalization after 20 minutes.

This parser deployment process is not required, once we successfully submit the parser to google repo.

View Events in Google SecOps

1. Log in to Google SecOps:
 - a. Open a web browser and navigate to the Google SecOps instance URL. For example: <https://test.backstory.chronicle.security/>
 - b. Replace test with your actual Google SecOps instance name.
2. Access SIEM Search:
 - a. From the top left corner of the Google SecOps console, select the "Investigation" option.
 - b. Within the Investigation section, choose "SIEM Search".
3. Filter Events by Log Type:
 - a. In the SIEM Search interface, locate the "UDM Search" section.
 - b. Apply a filter for the metadata field "log_type". Set the filter value to metadata.log_type="VECTRA_DETECT".
4. View Vectra Events:
 - a. The SIEM Search results will display Vectra events within the "Events" section.



The screenshot shows the Google SecOps SIEM Search interface. The search bar contains the query: `1 metadata.log_type = "VECTRA_DETECT"`. The results are displayed in a table with the following columns: **EVENTS**, **PIVOT**, **Q**, **Search events...**, **WRAP TEXT**, **COLUMNS**, and **...**. The table contains the following data:

TIMESTAMP	EVENT	USER	HOSTNAME	PROCESS NAME	SECURITY_RESULT.SUMMARY
2025-01-23T12:15:26.000	USER_RESOURCE_ACCESS unknown resource	[Unknown]	[Unknown]	[Unknown]	[Unknown]
2025-01-23T12:15:21.000	GENERIC_EVENT CREATED created	10188	[Unknown]	[Unknown]	[Unknown]
2025-01-23T12:15:21.000	GENERIC_EVENT DELETED deleted	10188	[Unknown]	[Unknown]	[Unknown]
2025-01-23T12:15:19.000	USER_RESOURCE_ACCESS unknown resource	[Unknown]	[Unknown]	[Unknown]	[Unknown]
2025-01-23T12:14:47.000	GENERIC_EVENT Detect	[Unknown]	[Unknown]	[Unknown]	[Unknown]

Environment Variables to be configured in Cloud Function

Variables	Description	Required	Datatype	Default Value	Secret
VECTRA_PORTAL_URL	Vectra portal URL	Yes	string	-	No
CLIENT_ID	Client Id generated from API Client	Yes	string	-	Yes
SECRET_KEY	Secret Key generated from API Client	Yes	string	-	Yes
CHRONICLE_CUSTOMER_ID	Google SecOps platform customer ID	Yes	string	-	No
CHRONICLE_REGION	Google SecOps platform region	No	string	us	No
CHRONICLE_SERVICE_ACCOUNT	Contents of the SecOps service account JSON file	Yes	string	-	Yes
GCP_BUCKET_NAME	GCP bucket name.	Yes	string	-	No
GCP_PROJECT_NUMBER	GCP project number where secrets are created.	Yes	string	-	No
INCLUDE_TRIAGED	Cloud function input parameter	No	bool	false	No
INCLUDE_SCORE_DECREASES	Cloud function input parameter	No	bool	false	No
INCLUDE_INFO_CATEGORY	Cloud function input parameter	No	bool	true	No
VLANS	Cloud function input parameter	No	bool	false	No

HISTORICAL	If set true, collect 24Hr Historical data.	No	bool	false	No
ENABLE_SCORING	Enables the ingestion and processing of scoring events.	No	bool	true	No
ENABLE_DETECTION	Enables the ingestion and processing of detection events.	No	bool	true	No
ENABLE_HEALTH	Enables the ingestion and processing of health events.	No	bool	true	No
ENABLE_AUDIT	Enables the ingestion and processing of audit events.	No	bool	true	No
ENABLE_LOCKDOWN	Enables the ingestion and processing of lockdown events.	No	bool	true	No

Looker Installation & Configuration

Pre-Requisites

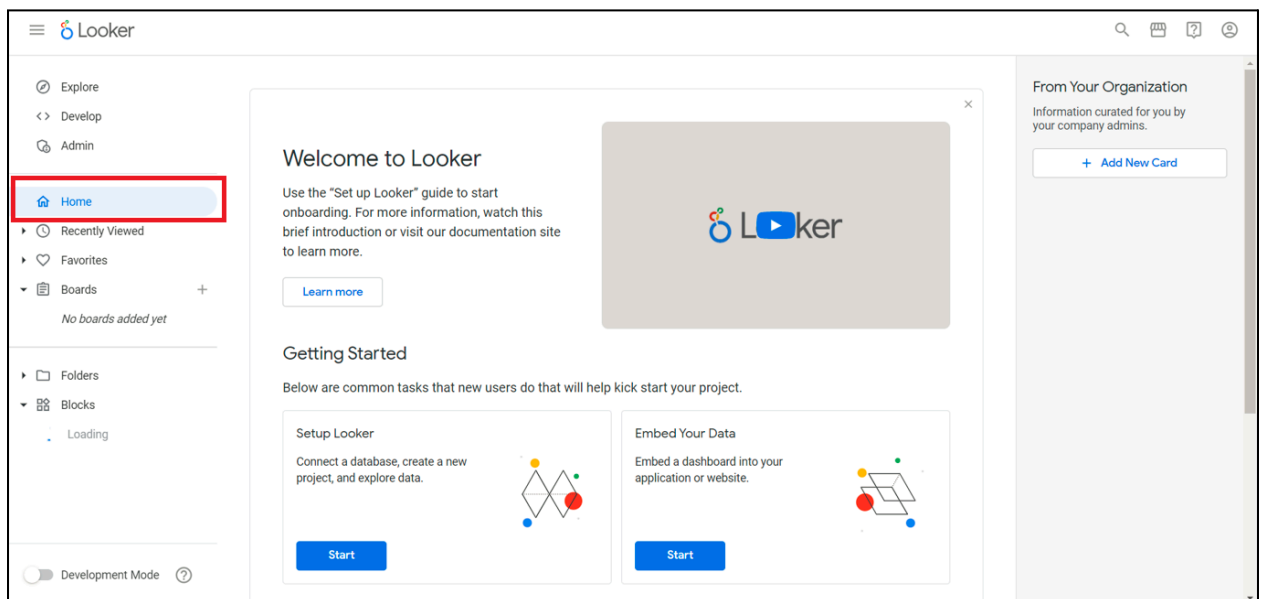
- Billing Project ID, Dataset name, and Service account file of BigQuery that stores Google SecOps data for database connection in Looker.
- BigQuery Export feature needs to be enabled for your Google SecOps tenant. (Reach out to your Google SecOps representative to set this up.)

User Permissions

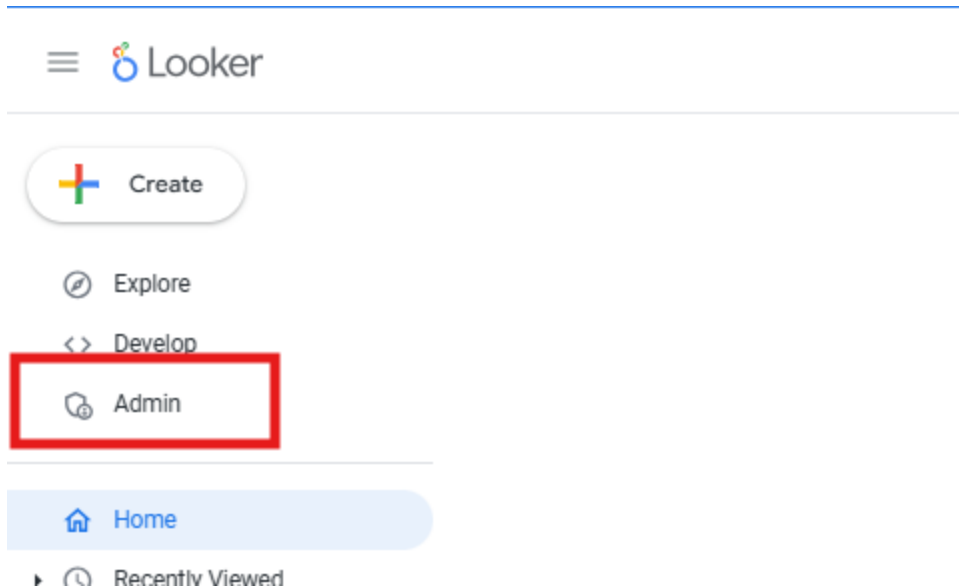
- Admin Role User - to create database connections and install blocks from the marketplace.

Create a connection to Google SecOps in Looker

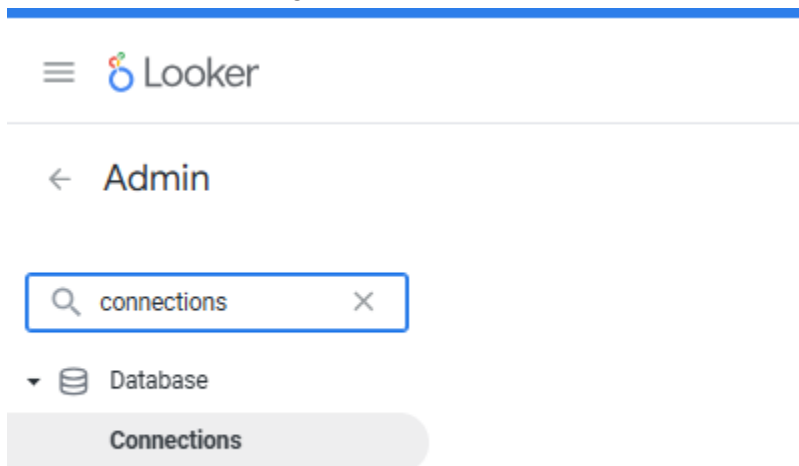
1. To create a connection to Google SecOps, first open the Looker instance and navigate to the Home page.



2. Now click on the **Admin** from the main menu (in left panel)



3. Now type **Connections** in the search, once the Connections option appears click on it to see the connection page.



4. Now click on the **Add connection**([Add Connection](#)) to create a new connection and name it as **chronicle**.
5. Select **Google BigQuery Standard SQL** in the Dialect. Now several new fields will appear.
6. Enter Billing Project ID field. Example: "**chronicle-crds**" here, where Chronicle data is present.
7. Enter the **datalake** in the Dataset name.

Connect your database to Looker

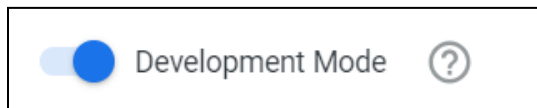
Fill out the connection details. The majority of these settings are common to most database dialects. [Learn more](#)

The screenshot shows the 'Connect your database to Looker' configuration page. It includes a 'Name' field with 'chronicle', a 'Connection Scope' section with 'All Projects' and 'Selected Project' buttons, a 'Dialect' dropdown set to 'Google BigQuery Standard SQL', a 'Billing Project ID' field with 'chronicle-crds', and a 'Dataset' field with 'datalake'. The 'Authentication' section has 'Service Account' and 'OAuth' buttons, with a text input for 'Upload service JSON or P12 file' and an 'Upload File' button. Below these is an 'Optional Settings' section with expandable options for 'SSH Tunnel', 'Persistent Derived Tables (PDTs)', 'Time Zone', and 'Additional Settings'. At the bottom are 'Test' and 'Connect' buttons.

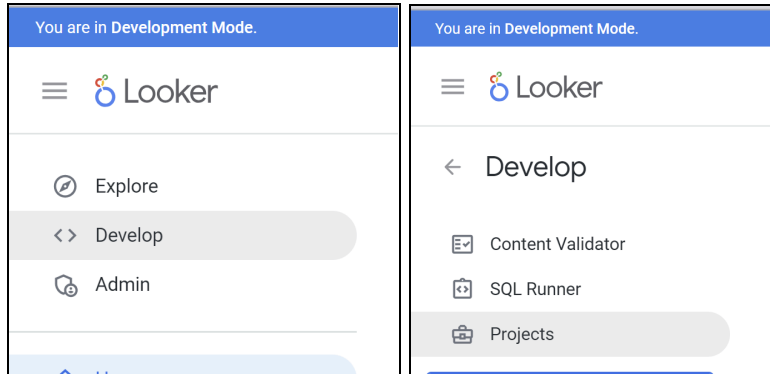
8. To configure authentication, select the service account method and upload your Chronicle service account file.
9. In the optional settings, set both the timestamps (Database timestamp and query timestamp) as UTC (the time fields shown in dashboards will be populated accordingly).
10. Click on Test to check the connectivity of Looker with Google Chronicle database.
11. Click on the Connect button () to complete the connection setup. Looker is now connected to the Google Chronicle database.

Get the Block from GitHub Repository

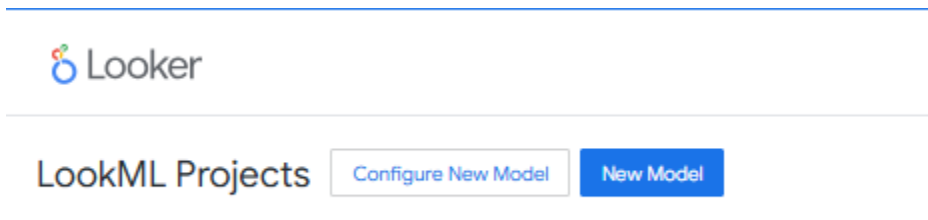
1. Go to Vectra looker dashboard github repository and fork the same. Make sure to uncheck the option for fork only the vectra_rux_dashboard branch.
2. Go to Looker and turn on "Development Mode" from the sidebar panel.



3. Select Projects from the Develop menu.



- From the LookML Projects page, select **Configure New Model** to open the model configuration page.



- Enter the model name as **vectra_rux_dashboard** and keep the **Same as Model** option selected.

Looker Browse ▾ Explore ▾ Develop ▾ Admin ▾

Configure a Model

Model

Project

- ☒ Same as Model
- ☐ Specify Project Name

Allowed Connections

- ☐ All (including any new connections added in the future)
- ☒ Only these connections:
 - ☐ 2059_chronicle_bigquery_connection (Instance wide)
 - ☐ 2059_postgresql_connection (Instance wide)
 - ☐ 2106_chronicle_bigquery_db (Instance wide)
 - ☐ 2106_mysql_db (Instance wide)
 - ☐ 2ba76e88c30c4b2492e0f3182c222d86 (Instance wide)
 - ☐ abcd (Instance wide)
 - ☐ am_0bf6b80e03f949ab841699a561944d90 (Instance wide)
 - ☐ am_2ba76e88c30c4b2492e0f3182c222d86 (Instance wide)
 - ☐ am_8274a6ee99a54cd4a0fc022762abf893 (Instance wide)
 - ☐ am_authmind_test3 (Instance wide)
 - ☐ am_authmind_test1 (Instance wide)
 - ☐ am_authmind_test2 (Instance wide)
 - ☐ am_authmind_test3 (Instance wide)
 - ☐ authmind (Instance wide)
 - ☐ authmind_api (Instance wide)
 - ☒ chronicle (Instance wide)
 - ☐ clickhouse_authmind (Instance wide)
 - ☐ corelight-chronicle (Instance wide)

- Click on **Save** to save the configuration.

- After saving the configuration you'll be redirected to the **Projects** page and you'll find your project in the **Pending Project** section.
- Click on **Add LookML** to configure your project and add the lookML file in it.



- On the New Project page, configure these options for your new project:
Project Name: Give project name 'vectra_rux_dashboard'.
Starting Point: Select Blank Project.
Click on Create Project. The project will be created and opened in the Looker IDE.

New Project

Project Name

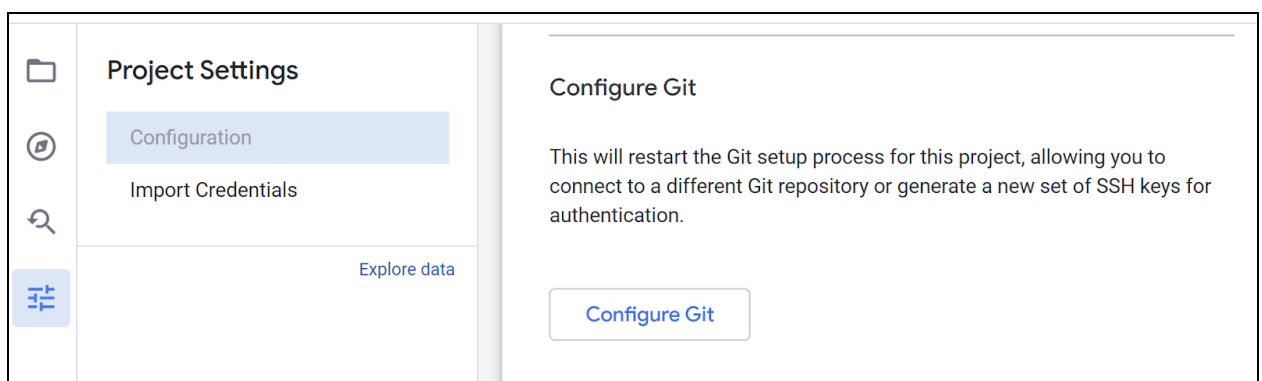
May contain lowercase letters, numbers, underscores, and dashes. Other characters will be lowercased or replaced with "_".

Starting Point

- ☐ Generate Model from Database Schema
- ☐ Clone Public Git Repository
- ☒ Blank Project

Create Project

- Click on the Settings icon from the navigation bar, and open the Configure Git page by selecting the Configure Git button.



- In Looker's Configure Git section, paste the URL of the forked [Vectra Looker Dashboard](#) Git Repository in the Repository URL field, then select Continue.

e.g. `https://github.com/<your_username>/looker-dashboards.git`

Configure Git

To configure Looker with Git, you'll need an existing empty Git repository hosted somewhere.

[How to Create a Repository](#)

Repository URL

The Repository URL should look something like
`git@github.com:myorganization/myproject.git` or
`https://github.com/myorganization/myproject.git` or
`ssh://git@github.com:22/myorganization/myproject.git`

After setting up Git, you can commit and deploy the models and dashboards in this project, making them explorable by other users.

You can choose which users can view models in the [user admin panel](#) after the the project has been deployed.

Don't have access to a Git server? [Set up a bare repository instead.](#)

[Continue](#)

12. Enter the github username and Personal Access Token, then click “Test and Finalize Setup”.

Note: Make sure the **Personal Access Token (PAT)** you created from your github repository has **Read/Write** permissions of the repository.

Configure Git

It looks like you're using https to connect a GitHub repository.

You're connecting to the repository [myorganization/forked-project](#).

Looker will authenticate with your GitHub repository using a username and personal access token. Please provide them below.

If you intended to connect without a personal access token (using a Deploy Key) please go back and provide a `git@...` style URL instead.

☒ Use a single, constant username and personal access token combination.

☐ Use user attributes for username and personal access token.

Username

Personal Access Token

[Test and Finalize Setup](#)

13. If you get an error like “Ensure credential allow write access failed”, just enter the username and token again and click “Skip Tests and Finalize Setup”.

☒ Use a single, constant username and personal access token combination.

☐ Use user attributes for username and personal access token.

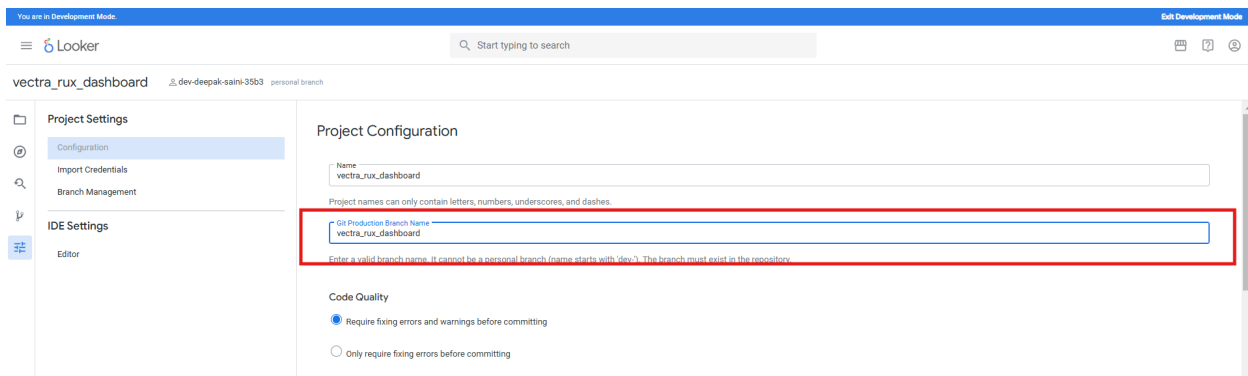
Username

Personal Access Token

Ensure credentials allow write access failed
HTTPS credentials do not have write access. (Is a 2 factor auth token required?)

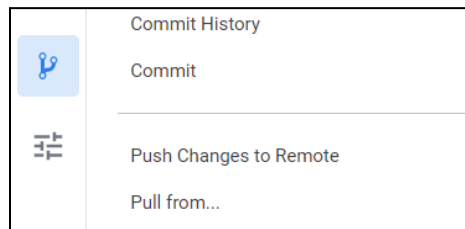
[Test and Finalize Setup](#) [Skip Tests and Finalize Setup](#)

14. Once the git is configured, open the project settings and change the production branch to **vectra_rux_dashboard**

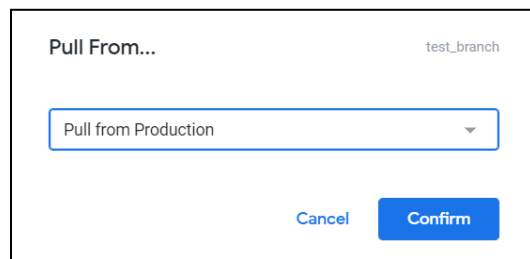


15. Now, you should be able to see the code in your project from the **vectra_rux_dashboard** branch. If not then,

- a. In the 'Git Actions' tab from the left side, click on the "Pull from..." option.



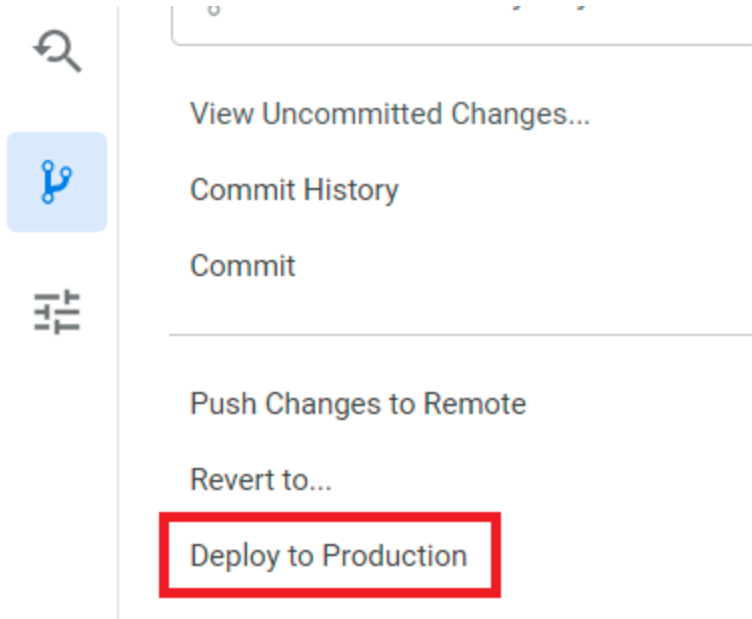
- b. Select the "Pull From Production" option and click on the Confirm button.



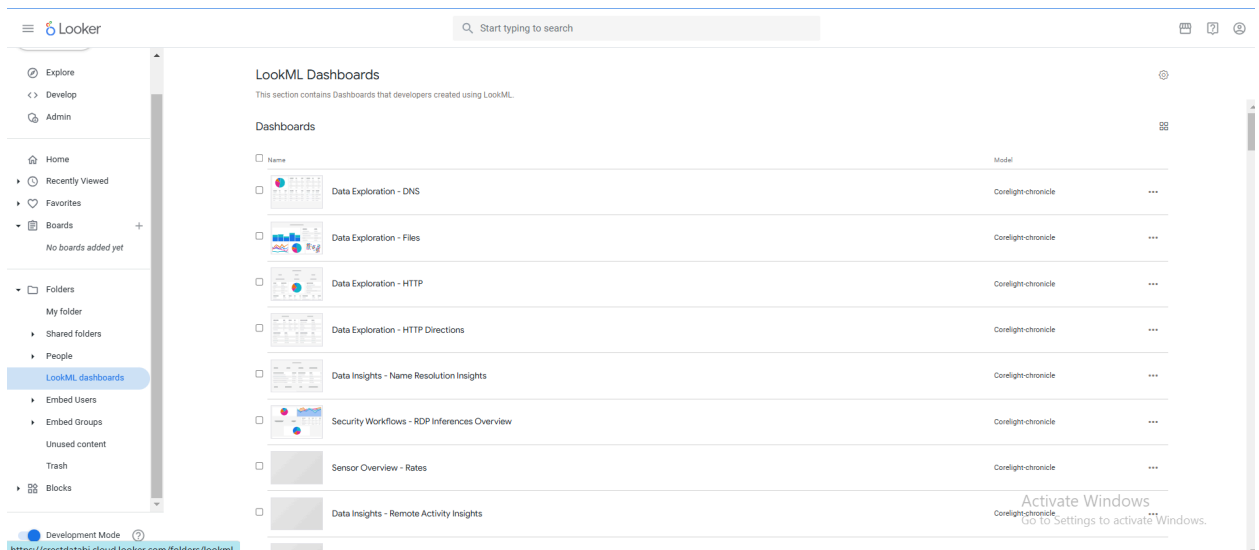
16. Any changes can be committed and pushed to the dev branch by clicking on **Validate LookML** and once the files are validated click on **Commit and Push changes**.

17. After the above steps, In the Git Actions, click on the "Deploy to Production" or you can press "Deploy to Production" from the top right corner.

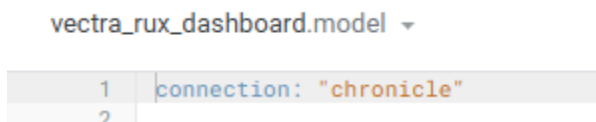
Note: 'Deploy to Production' will push code to the production branch that is set in the project settings. By default, it will be the vectra_rux_dashboard branch. If you don't want to push code to the 'main' branch, then create your own branch and set it to 'Git Production Branch Name' in project settings. Then click on Deploy to Production.



18. Now you can turn off the development mode in order to see the LookML dashboards.
19. On the Homepage of your Looker instance, navigate to the “LookML dashboards” tab under the “Folders” tab to access and view all the dashboards.



20. The connection name defined at the top of the **vecetra_rux_dashboard.model** file must match the connection name created earlier. If the user has named it chronicle, no changes are necessary. Otherwise, the connection field needs to be updated with the correct connection name.



Dashboards

Entities Dashboard

This dashboard displays data for the "Scoring" log type, including incidents that are either prioritized or not. The table panel presents details about each incident, including their most recent update status.

1. Filters description as per label

Log type

- This filter contains a single value for the dashboard, and other filters will update based on the selection made in the **Log Type** filter. **Default:** Scoring.

Timerange

- This filter updated the panel based on the time range selected in it. **Default:** Last 7 days.

Entity Type

- Filters the Panels according to the selected entity type **i.e.** account or host. **Default:** all.

Prioritized

- Filters the Panels according to the selected priority **i.e.** true or false. **Default:** all.

Data Source Type

- Filters the Panels according to the selected data source type. Filter have the following values AWS, O365, M365, SAML and Network **Default:** all.

2. Panel description as per label

Prioritized

- The single value panel displays the count of prioritized incidents based on their most recent updates, which were collected from the Vectra RUX platform sent to Google SecOps.

Not Prioritized

- The single value panel displays the count of unprioritized incidents based on their most recent updates, which were collected from the Vectra RUX platform sent to Google SecOps.

Entities List

- The table panel displays the incidents of prioritized and unprioritized incidents based on their most recent updates, which were collected from the Vectra RUX platform sent to Google SecOps.

Log Type 

Scoring  

Timerange

is in the last 2 days

Entity Type 

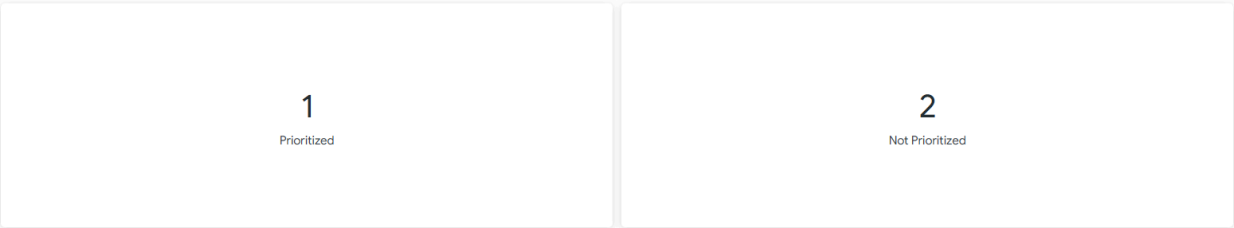
any value 

Data Source Type 

any value 

Prioritized

any value 



Entities List										
	Urgency Score	Attack Rating	Entity Name	Vectra Pivot	Prioritized	Data Source Type	Importance	Velocity	Last Updated	
1	53	5	g.....	Pivot to vectra	false	SAML	Medium	Low	2025-01-22 07:40:28	
2	40	8	Importance Velocity Test	Pivot to vectra	true	Network	Unknown	Unknown	2025-01-22 06:30:00	
3	32	1	g.....	Pivot to vectra	false	D365	Medium	Low	2025-01-22 06:35:20	

Detection Dashboard

1. Filters description as per label

Log type

- This filter contains a single value for the dashboard, and other filters will update based on the selection made in the **Log Type** filter. **Default:** Detection.

Timerange

- This filter updated the panel based on the time range selected in it. **Default:** Last 7 days.

Data Source Type

- Filters the Panels according to the selected data source type. Filter have the following values AWS, O365, M365, SAML and Network **Default:** all.

Behaviour

- Filters the Panels according to the selected behaviour. **Default:** all.

Category

- Filters the Panels according to the selected category. **Default:** all.

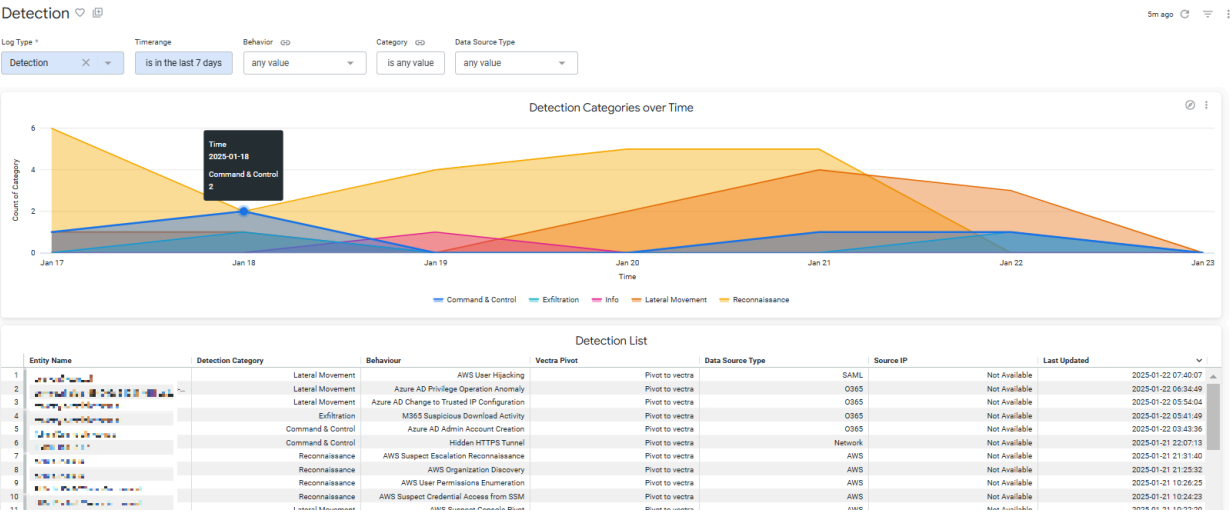
2. Panel description as per label

Detection Categories over Time

- This displays an area chart for the count of the categories over time of Detection log.

Detection List

- The table panel displays the incidents of detection logs based on their most recent updates, which were collected from the Vectra RUX platform sent to Google SecOps.



Audit Dashboard

1. Filters description as per label

Log type

- This filter contains a single value for the dashboard, and other filters will update based on the selection made in the **Log Type** filter. **Default:** Audit.

Timerange

- This filter updated the panel based on the time range selected in it. **Default:** Last 7 days.

Username

- Filters the Panels according to the selected username. **Default:** all.

Status

- Filters the Panels according to the selected status. **Default:** all.

2. Panel description as per label

Audit Table

- The table panel displays the incidents of audit logs based on their most recent updates, which were collected from the Vectra RUX platform sent to Google SecOps.

Audit  

Log Type  Timerange  Username  Status 

Audit  is in the last 7 days any value any value

	Event Timestamp	Username	Role	Status	Message	
1	2025-01-22 08:42:51			Security Analyst	success	log in
2	2025-01-21 06:15:15			Security Analyst	success	log in
3	2025-01-20 06:14:39			Security Analyst	success	log in
4	2025-01-19 16:14:54			Security Analyst	success	log in

Lockdown Dashboard

1. Filters description as per label

Log type

- This filter contains a single value for the dashboard, and other filters will update based on the selection made in the **Log Type** filter. **Default:** Lockdown.

Timerange

- This filter updated the panel based on the time range selected in it. **Default:** Last 7 days.

Entity Type

- Filters the Panels according to the selected entity type **i.e.** account or host. **Default:** all.

Is Locked

- Filters the Panels according to the selected “Is locked” **i.e.** true or false. **Default:** all.

2. Panel description as per label

Lockdown

- The table panel displays the incidents of lockdown logs based on their most recent updates, which were collected from the Vectra RUX platform sent to Google SecOps. It shows whether the incident is still locked or not.

Lockdown ▾ ⓘ

24m ago 🔍 ⌵ ⌵ ⌵

Log Type * Timerange Entity Type Is Locked

Lockdown × Is in the last 7 days any value any value

	Entity name	Locked Date	Locker By	Unlock Date	Entity Locked	Entity Type
1		2025-01-21 10:18:00		2025-01-20 15:06:23	False	host
2		2025-01-21 10:18:00		2025-01-20 15:06:35	False	account
3		2025-01-21 10:18:00		2025-01-20 15:07:02	False	host
4		2025-01-21 08:06:23		2025-01-21 15:06:23	False	host
5		2025-01-21 08:06:23		2025-01-21 15:06:23	False	host
6		2025-01-20 14:07:02		2025-01-20 15:07:02	False	host
7		2025-01-20 14:06:35		2025-01-20 15:06:35	False	account
8		2025-01-20 14:06:23		2025-01-20 15:06:23	False	host
9		2025-01-17 14:10:09		2025-01-18 14:10:09	False	account
10		2025-01-17 14:10:09		2025-01-18 14:10:09	False	account

Health Dashboard

1. Panel description as per label

System Table

- This panel displays the most recent data of the system's resource utilization.

Sensors Table

- This panel displays the most recent data related to sensors like connectivity status, connectivity error, trafficdrop status, trafficdrop error etc.

System Table									
System Version	Last Update	CPU Usage - User(%)	CPU Usage - System(%)	CPU Usage - Idle (%)	Disk Utilization (%)	Memory Usage (%)	Aggregated Peak Traffic (Mbps)	Power Status	Power Error
1	Thu Dec 5 23:29:58 2024	27.8	18.5	52.2	36.2	52.5	0	SKIP	Power check for this device...

Sensor Table								
	Name	IP Address	Connectivity Status	Connectivity Error	Trafficdrop Status	Trafficdrop Error	Aggregated Peak Traffic (Mbps)	Link Status
1	EDR Sensor	10.10.10.10	OK	metadata replication seems fine	OK	All interfaces have traffic volum...	3	UP

Notes

- If an optional environment variable is not provided during the Cloud Function deployment, default values will be used, and data collection will start accordingly.
- The chunk limit for data collection is set to 100 to minimize data duplication in case of errors during ingestion, as the Google SecOps Ingestion API processes data in chunks of 100.
- We recommend setting the timeout in the RUNTIME variable to the maximum value (3600) to prevent the Cloud Function from terminating during data collection.
- The dashboard displays data in a tabular format and will show only the top 100 incidents, sorted by the selected field in the table.
- The Data Source Type will no longer include static values like Network, Microsoft 365 / Azure AD, and AWS. Instead, it will feature AWS, M365, O365, SAML, and Network, and filter data accordingly.

Limitations

- If the user does not specify the required environment variable while configuring the Cloud Function, the script deployment will fail.
- CBN parser will only be able to parse the Vectra RUX events.
- We suggest using the second generation of Cloud Function. The first generation of Cloud Function has a maximum execution time of 9 minutes and the second generation of Cloud Function has a maximum execution time of 60 minutes. If the execution time of the Cloud Function exceeds timeout then there are chances that the complete data won't be ingested in the Google SecOps.
- The rate limit for a Vectra account depends on the user's subscription. Based on this API rate limit, the integration will be able to collect data and ingest into Google SecOps. Once the API rate limit is exceeded, data collection will only resume when the limit is reset after a specific interval.
- Looker doesn't support API calls to fetch live data for populating dashboards.
- We do not have any marketplace for Google SecOps integration, so as a part of deliverables, we use to push our integration code to google public repo which undergoes many checks from google side.
- Common search filters can't be implemented in looker.
- Pagination in tabular visualization is not supported in Looker.
- Looker will only show data from the past 180 days, but this can vary as per the retention policy configured in BigQuery.

- According to the query time zone selected by the user in connection with the Google SecOps database, the Looker dashboards would be reflected according to the configured timezone.
- The looker dashboard does not display data in the drill down table when there are too many records to be displayed.

Troubleshooting

This section describes the common issues that might happen during the deployment or the running of the app and the steps to resolve the issues.

1. GCloud logs can be used for troubleshooting.
 - a. Log in to the "https://console.cloud.google.com/" using valid credentials.
 - b. Navigate to 'Cloud functions' and click on the deployed function where you can find the logs module.
 - c. Logs can be filtered using severity.

Currently, this logs feature is disabled by the google team in some GCP projects. We are currently checking with the google team regarding this.

2. If you test the cloud function immediately after deploying it on gcloud, It might be possible that the cloud function will not work as expected. To resolve this, wait for a few seconds and then test it.
3. If the cloud function stops its execution because memory exceeds the limit, reconfigure the cloud function's memory configuration and increase the memory limit.
4. The Looker dashboard takes data from the cache and does not display the latest events.

Solution:

1. Click on the three dots present on the rightmost side of the dashboard.
2. Click on the Clear cache and refresh.

Health  4m ago  

System Table								
	System Version Last Update	CPU Usage - User(%)	CPU Usage - System(%)	CPU Usage - Idle (%)	Disk Utilization (%)	Memory Usage (%)	Aggregated Peak Traffic (Mbps)	Power Stat
1	Thu Dec 5 23:29:58 2024	27.8	18.5	52.2	36.2	52.5	0	SKIP

Sensor Table							
	Name	IP Address	Connectivity Status	Connectivity Error	Trafficdrop Status	Trafficdrop Error	Aggregated Peak T
1	EDR Sensor	192.168.1.100	OK	metadata replication seems fine	OK	All interfaces have traffic volum...	3

 Clear cache and refresh 
Show dashboard details
Dashboard performance summary
< > Go to LookML
Copy LookML dashboard
Get aggregate table LookML
Download  alt:D
Schedule delivery  alt:S
Add to a board
Get embed URL
Get link

5. The data is not displayed on the dashboard -
This could be a problem with the data source as the database connection might be wrongly configured.
6. If desired events are not showing in the visualization -

Make sure that the filters in the dashboard are configured correctly. If the filters are too restrictive, they may be preventing the dashboard from displaying any Data.

7. The dashboard may be slow to load or unresponsive - This could be due to a problem with the data source being unavailable or having too much data, the query that is being used, or the way that the dashboard is being rendered.

GCP Resources/Services Approximate Cost Details

Service	Standard Configurations	Purpose	Reference
Cloud Functions	Type: Memory: 8192MB CPU: 4.8GHz Execution time per function (ms): 3600 Invocations per month: 1500 Minimum number of instances: 1	Function / Script which pulls data from Vectra Platform using API and ingest into Google SecOps.	Approx cost ~ \$66/month https://cloud.google.com/functions/pricing
Cloud Storage	Total Amount of Storage: 1 GiB	Storage bucket used to manage API checkpoints	Approx cost ~ \$0.02/month https://cloud.google.com/storage/pricing
Secret Manager	Access operations: 1500	Used to maintain credentials.	Approx cost ~ \$0/month https://cloud.google.com/secret-manager/pricing
Cloud Scheduler	Total amount of jobs: 1	Scheduler which executes above cloud function at a specific time interval.	Approx cost ~ \$0/month https://cloud.google.com/scheduler/pricing
Looker	https://cloud.google.com/looker/pricing#platform_editions	Visualization tool to visualize events from Google SecOps.	https://cloud.google.com/looker/pricing Cost depends on the edition which we select from the link on the left column.

Note: Users can also calculate (using [pricing calculator](#)) the estimated price of the Google Cloud services used.

References

- ❖ [Install the gcloud CLI](#)
- ❖ [Deploying cloud functions from local machine](#)
- ❖ [Looker](#)
- ❖ [Looker Marketplace](#)